

# Engineering for Evolution

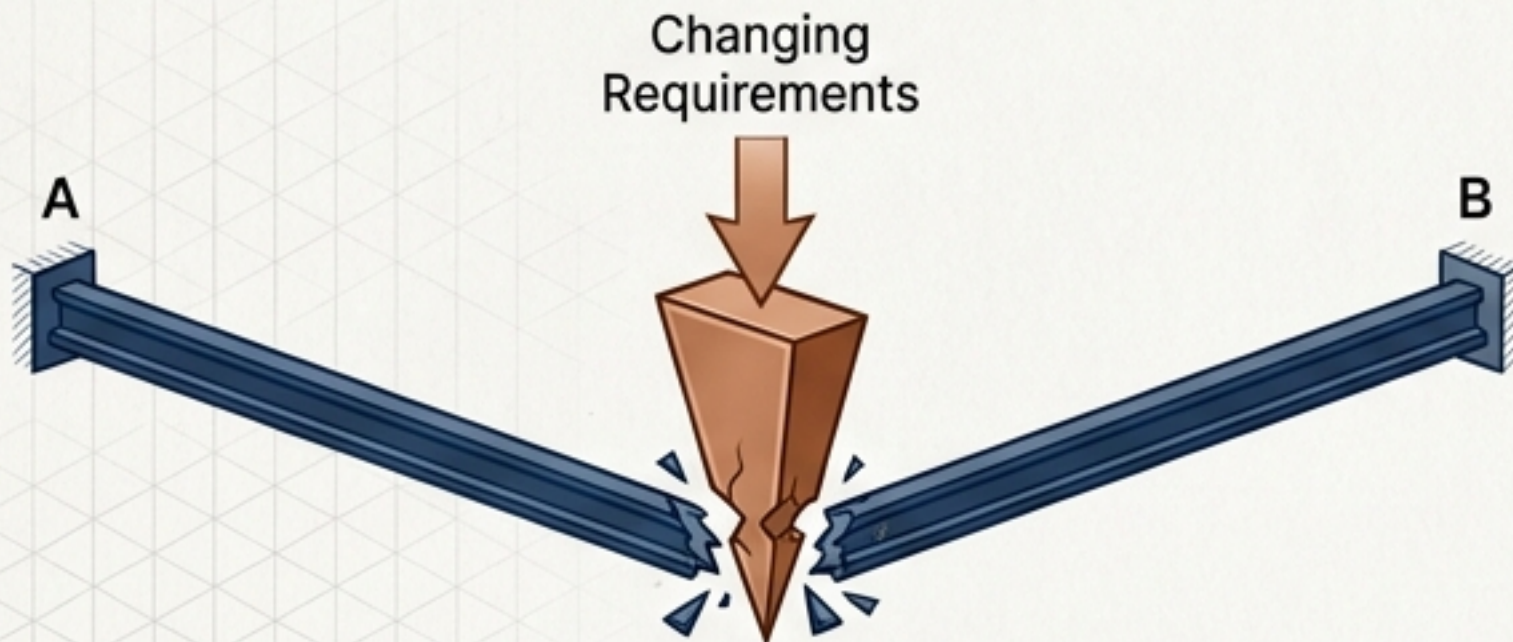
Navigating Change and Process Improvement  
in Software Development



STRATEGIC SUMMARY //  
PROCESS ARCHITECTURE

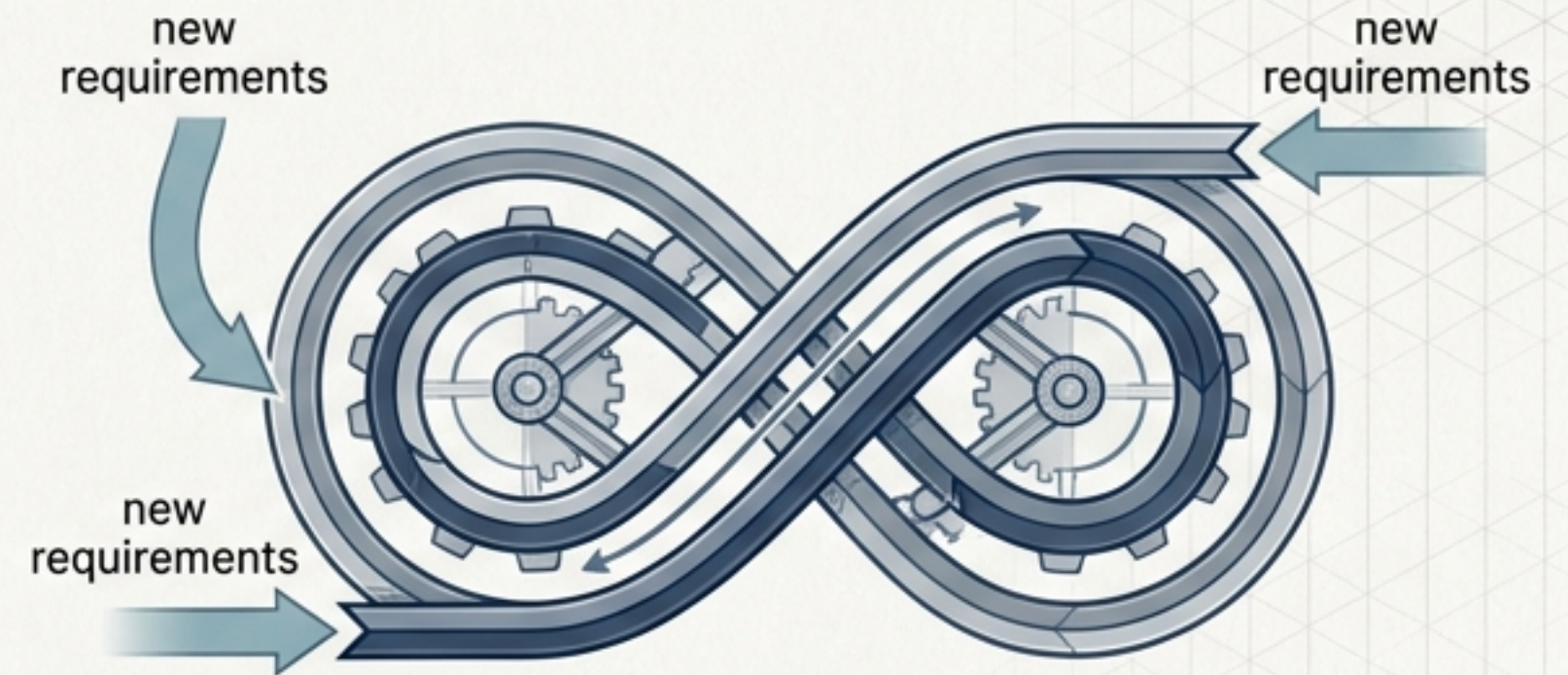
# Software is an Engine, Not a Monument

## Static Delivery



### The Myth

## Continuous Evolution



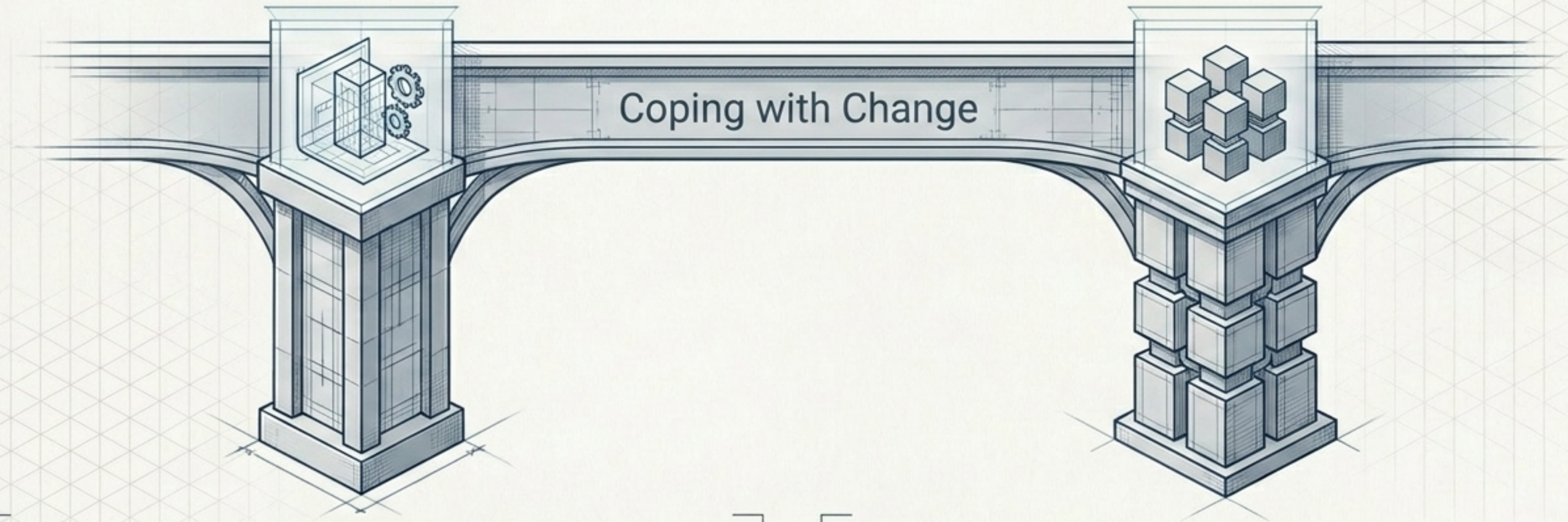
### The Reality

**Core Concept:** Software systems must evolve to remain useful. Developing software is a continuous negotiation with change, rendering linear processes ineffective.

System requirements are rarely static; users discover new needs as they interact with initial features.

Processes must be structurally designed to either anticipate or tolerate change to avoid disrupting the system as a whole.

# Two Strategic Pillars for Coping with Change



## Pillar 1: Change Anticipation (System Prototyping)

**Definition:** Developing a rapid, early version of the system (or part of it).

**Mechanism:** Allows users to experiment and refine requirements before delivery, filtering out bad proposals early.

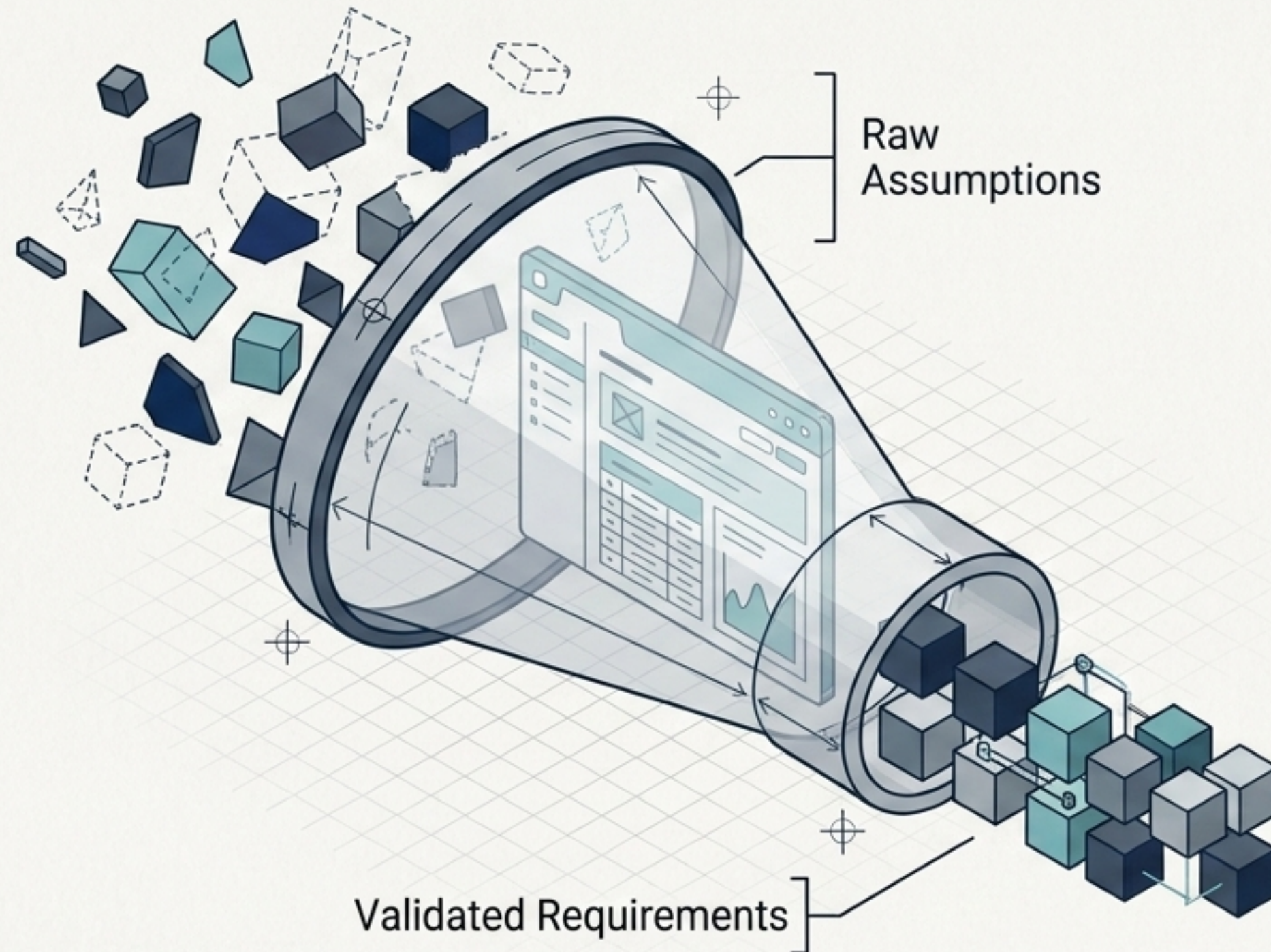
## Pillar 2: Change Tolerance (Incremental Delivery)

**Definition:** Delivering the system in prioritized increments.

**Mechanism:** Avoids premature commitment to the whole system, allowing changes to be incorporated into later stages at relatively low cost.

# System Prototyping: The Validation Filter

*Rapid, iterative prototype development controls costs by revealing errors and omissions early.*

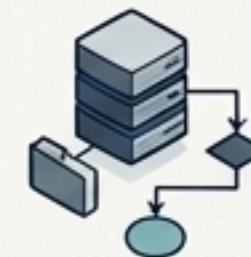


## Diagnostic Callouts (When to Prototype)



### Requirements Engineering

For elicitation and validation. Clarifies features that seem useful in isolation but fail when combined with other functions.



### System Design Feasibility

For running design experiments (e.g., testing if a database design supports efficient data access for common queries).



### User Interface (UI) Design

Textual descriptions are insufficient for dynamic UI; rapid end-user involvement is the only sensible approach.

# The Prototyping Pipeline



## ! The Rule of Reduction

To control costs and accelerate delivery, intentionally leave things out:

- Relax non-functional requirements (response time, memory).
- Ignore error handling (unless testing UI).
- Reduce standards of program quality and reliability.

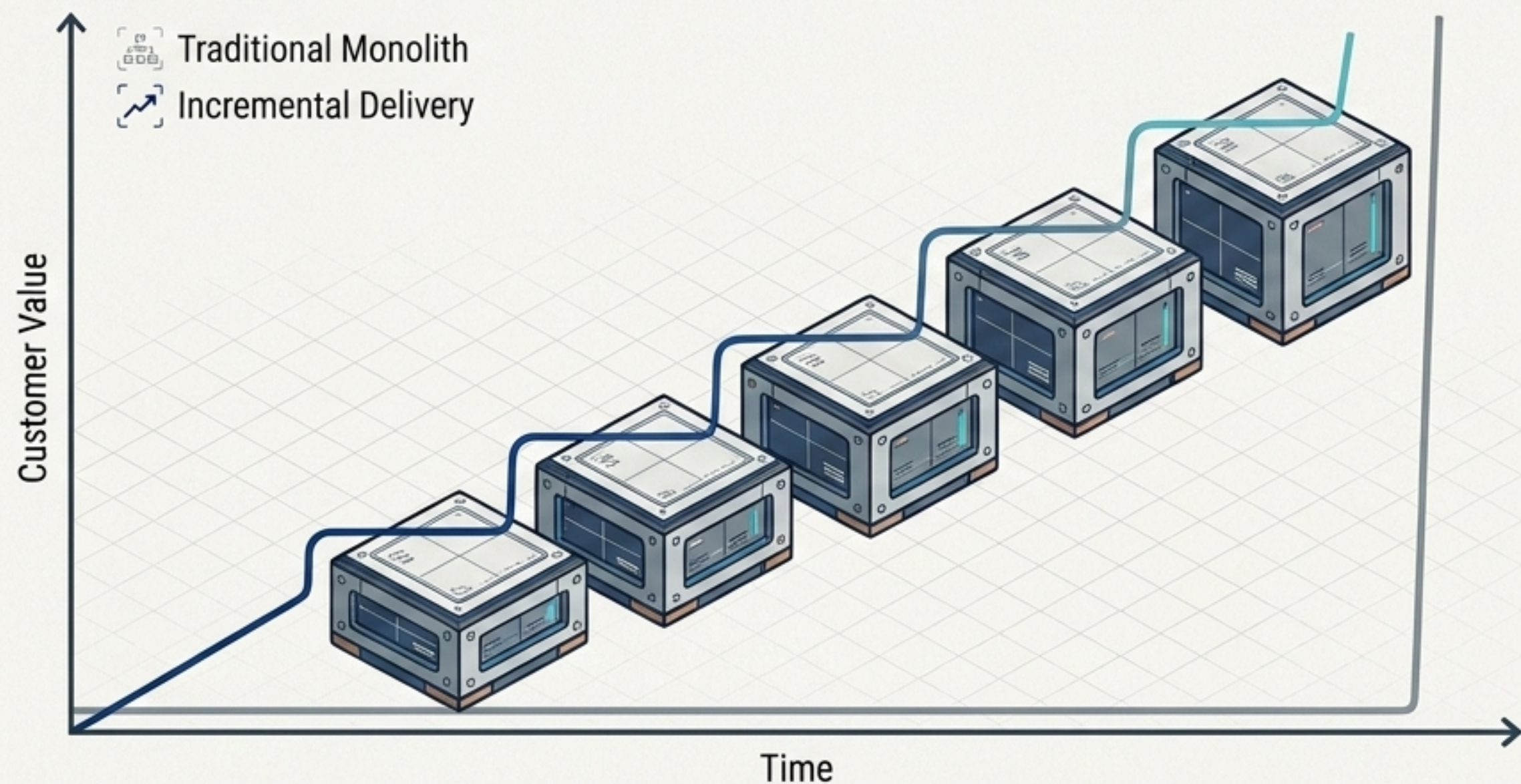
## The Core Risk

Evaluators may adjust their behavior around prototype flaws (e.g., avoiding slow features), leading to inaccurate usage data compared to the final system.

# Incremental Delivery: Compounding Value

Deploying developed increments into the customer's working environment based on strict priority.

## The Activation Curve

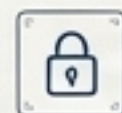


## The Execution Strategy



### Prioritize Services

Customers define which services are most critical. Highest priority dictates the first increment.



### Lock the Increment

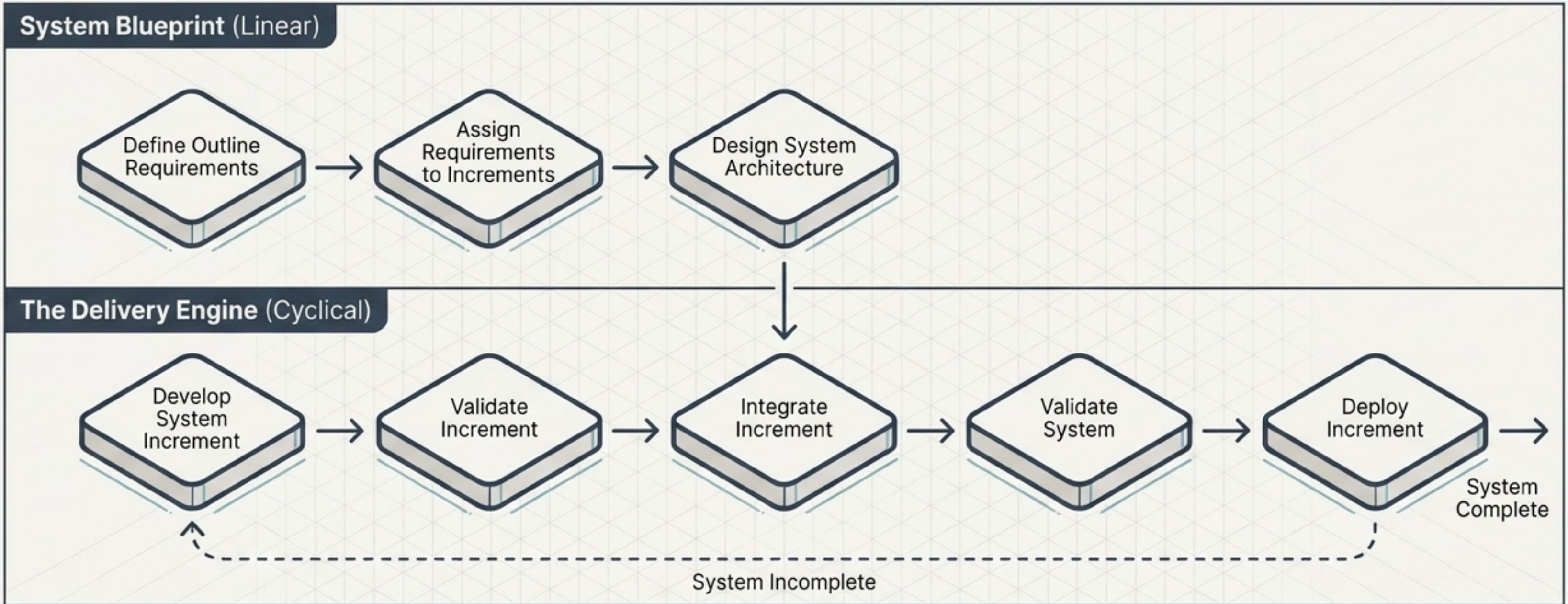
During development, requirements for the current increment are locked. Requirements analysis for future increments continues.



### Integrate & Advance

Customers gain immediate value. Their real-world experience clarifies requirements for the next increment.

# The Incremental Workflow



## Key Advantage

The most critical system services (Increment 1) are integrated first and receive the most testing across the entire lifecycle, drastically reducing failure rates in core components.

# The Reality of Incremental Delivery

## Tailwinds

**Real-System Experience:** Functions like a prototype, but uses the real system (no relearning required).

**Early ROI:** First increment satisfies the most critical requirements immediately.

**High Robustness:** Core features receive compounding testing over multiple increment cycles.

## Headwinds

**The Legacy Trap:** Unsuitable for replacing existing systems where users demand total feature parity on day one.

**Architecture Ambiguity:** Hard to identify common, shared facilities early when full requirements are undefined.

**Procurement Friction:** Conflicts with traditional contracts that demand complete system specifications upfront.

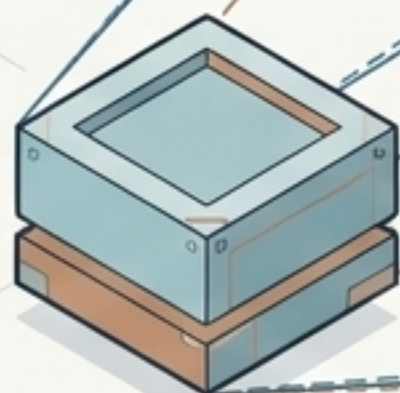
*Not recommended for: Geographically distributed mega-projects, hardware-dependent embedded systems, or highly-coupled safety-critical systems.*

# Framework Synthesis: Anticipate vs. Tolerate

|                     | System Prototyping                                 | Incremental Delivery                                    |
|---------------------|----------------------------------------------------|---------------------------------------------------------|
| Primary Objective   | Anticipate and filter out bad requirements.        | Tolerate change and deliver early value.                |
| Artifact Status     | A simulation; often discarded or heavily reworked. | The real system; built for permanent production.        |
| Customer Experience | Evaluators testing an abstraction.                 | Real users extracting business value.                   |
| Requirement Lock-in | Wholly fluid; designed to provoke changes.         | Rigid for the current increment; fluid for future ones. |

# Upgrading the Engine: Process Improvement

Software Increment  
(Modular Block)



Organizational  
Process Blueprint  
(The Engine)

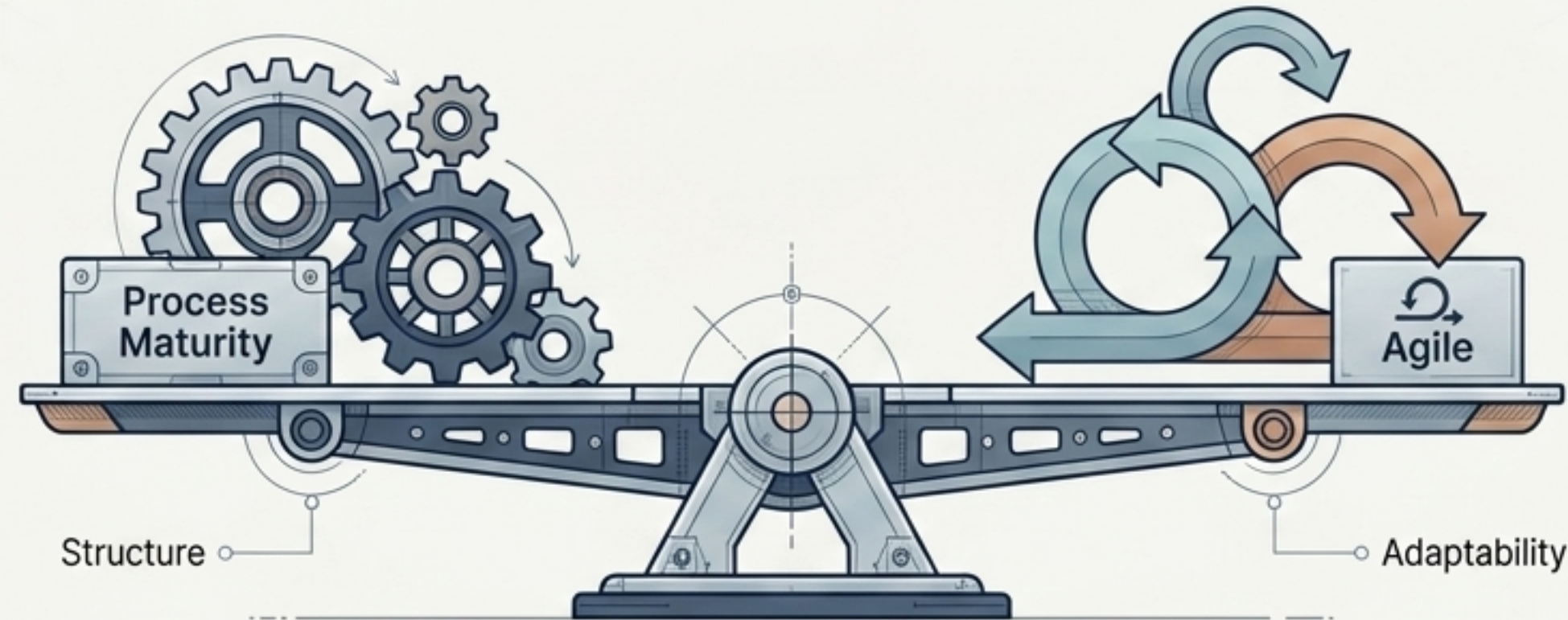
## Definition

Process improvement requires understanding existing processes and actively engineering them to increase product quality, reduce costs, and accelerate development.

## Core Concept

Mastering change at the project level is insufficient. The industry demands cheaper, better software delivered to ever-tighter deadlines. To survive, organizations must elevate their focus from the software to the machinery building the software.

# The Competing Philosophies of Improvement



## Philosophy 1: The Process Maturity Approach

- ⚙️ - Rooted in plan-driven development.
- ⚙️ - Focuses on organizational standardization, predictability, and improved product quality.
- ⚙️ - **Trade-off:** Requires high “overhead” (activities not directly relevant to writing code, like documentation and formal measurement).

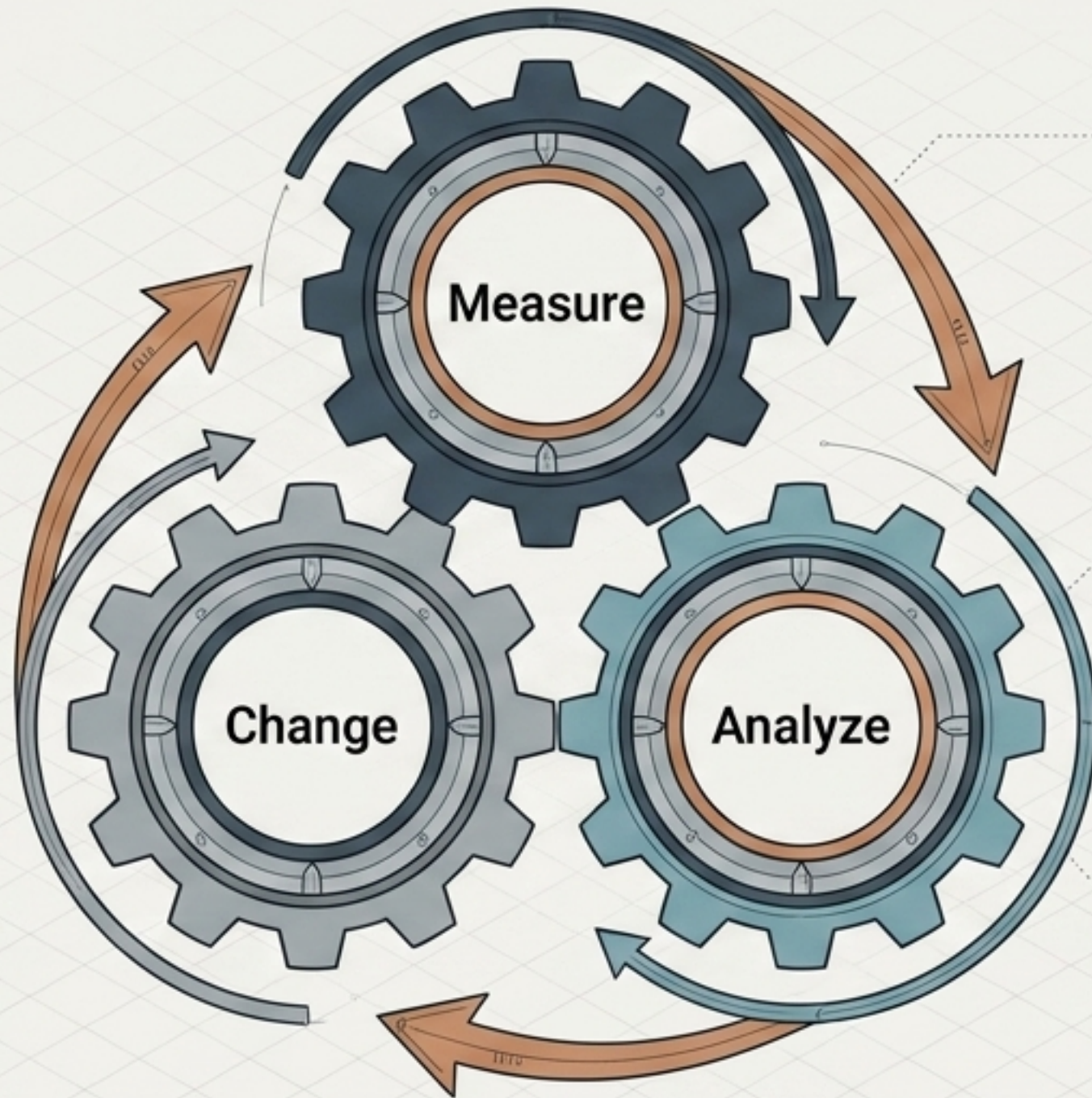
## Philosophy 2: The Agile Approach

- 🔄 - Focuses on iterative development, rapid delivery, and supreme responsiveness to changing customer needs.
- 🔄 - Assumes the best processes are those with the absolute lowest overhead.
- 🔄 - **Trade-off:** Deliberately minimizes formality and documentation.

## Insight Callout: The Cultural Divide

Enthusiasts of each approach are deeply skeptical of the other, viewing the opposition as either chaotic (Agile) or bureaucratic (Maturity).

# The Continuous Improvement Flywheel



## Measure

Establish a baseline. Measure specific attributes of the software process or product. Without concrete data, assessing improvement is impossible.

## Analyze

Assess the current process to identify bottlenecks, weaknesses, and map the workflow (focusing on rapidity and robustness).

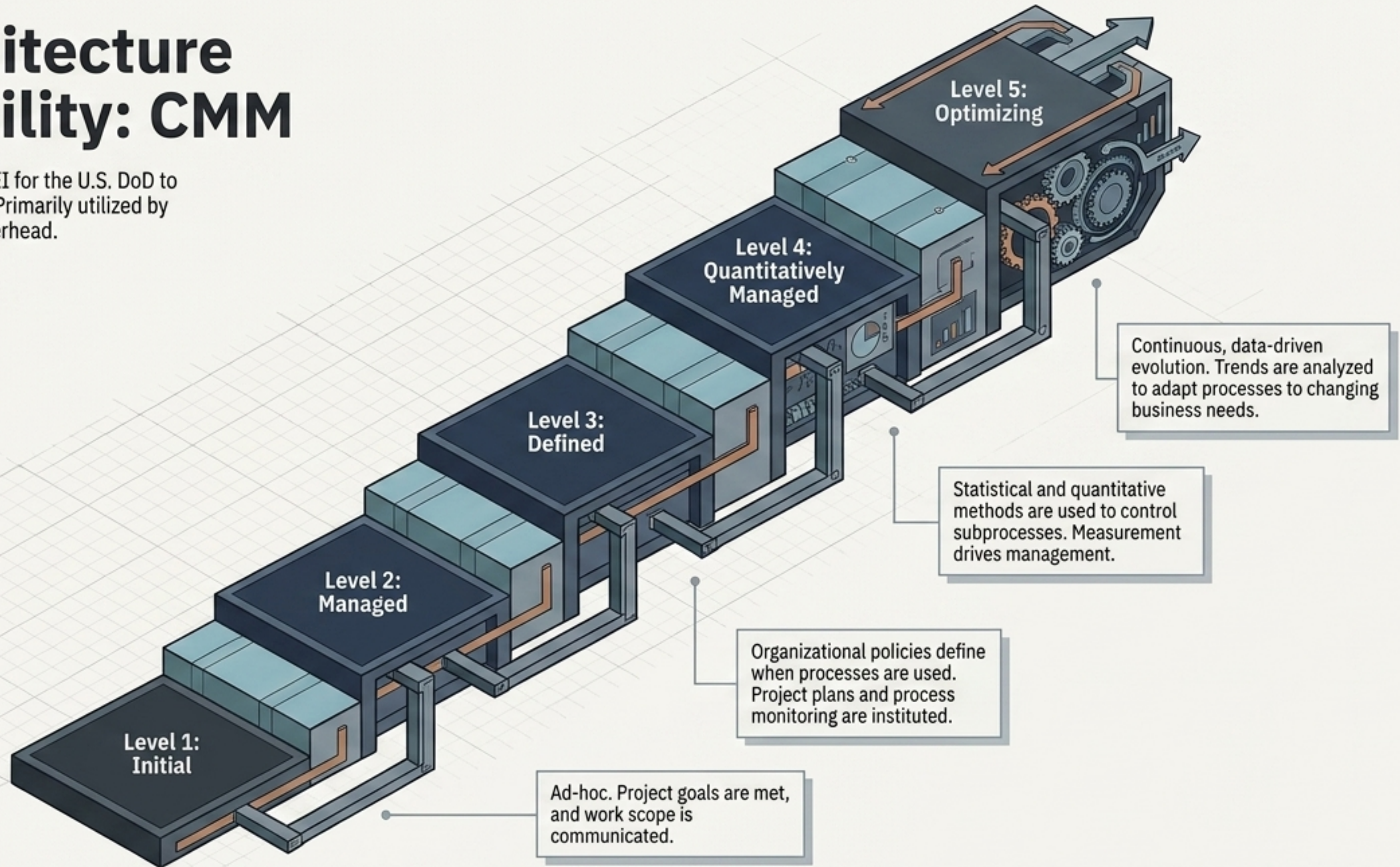
## Change

Introduce targeted process changes to address identified weaknesses, then resume the cycle to measure effectiveness.

**Strategic Context:** This is a long-term, continuous momentum engine. Because the business environment is always changing, the process must permanently evolve to keep pace.

# The Architecture of Capability: CMM

Context: Introduced by the SEI for the U.S. DoD to assess contractor capability. Primarily utilized by large organizations due to overhead.



# The Process Paradigm: A Summary



“Build the system to survive change; build the organization to engineer it.”